



Des interfaces aux Streams API

Corrigé des exercices

Public : L2 Informatique

Document lié : Exercices « Des interfaces aux Streams API »

Environnement : Java 17 (records, var)

Année universitaire : 2025-2026

Avant-propos

Ce corrigé reprend les cinq exercices de la série « Des interfaces aux Streams API ». Chaque solution respecte les consignes de l'énoncé : une classe ExerciceN avec une méthode main, aucune boucle for/while là où un Stream fait le travail, et aucun Optional[...] laissé à l'affichage. Tous les programmes ont été compilés (javac) et exécutés (java) réellement; les sorties reproduites ci-dessous sont les sorties effectivement obtenues, pas des sorties reconstituées.

Remarque sur les données

L'énoncé laisse le choix des données de départ. Les jeux de données utilisés ici (noms, notes, produits, étudiants) sont donc un exemple parmi d'autres possibles — seule la démarche Stream compte pour la correction.

1. Exercice 1 — Interface fonctionnelle et lambdas

Énoncé

Créer une interface fonctionnelle Operation avec une méthode int appliquer(int a, int b), puis quatre lambdas : addition, soustraction, multiplication, maximum. Résultat attendu pour $a = 7$, $b = 3$: 10, 4, 21, 7.

Solution

Une interface fonctionnelle ne contient qu'une seule méthode abstraite (annotée @FunctionalInterface pour que le compilateur le vérifie); chaque lambda (a, b) -> ... en devient une implémentation différente, sans écrire quatre classes.

```
1 import java.util.function.BinaryOperator;
2
3 public class Exercice1 {
4
5     @FunctionalInterface
6     interface Operation {
7         int appliquer(int a, int b);
8     }
9
10    public static void main(String[] args) {
11        Operation addition = (a, b) -> a + b;
12        Operation soustraction = (a, b) -> a - b;
13        Operation multiplication = (a, b) -> a * b;
14        Operation maximum = (a, b) -> Math.max(a, b);
15
16        int a = 7;
17        int b = 3;
18
19        System.out.println("addition      : " + addition.appliquer(a, b));
20        System.out.println("soustraction   : " + soustraction.appliquer(a, b));
21        System.out.println("multiplication : " + multiplication.appliquer(a, b));
22        System.out.println("maximum       : " + maximum.appliquer(a, b));
23    }
24 }
```

Sortie obtenue

```
addition      : 10
soustraction  : 4
multiplication : 21
maximum       : 7
```

À retenir

Les quatre résultats correspondent exactement à ceux annoncés par l'énoncé : 10, 4, 21, 7. Une interface fonctionnelle + lambdas remplace ici quatre classes concrètes qui auraient chacune implémenté `Operation`.

2. Exercice 2 — Streams sur une liste de chaînes

Énoncé

Sur une liste de noms : afficher tous les noms, afficher ceux qui commencent par A, les transformer en majuscules, les trier par ordre alphabétique, compter ceux de plus de 5 caractères.

Solution

```
1 import java.util.List;
2
3 public class Exercice2 {
4     public static void main(String[] args) {
5         List<String> noms = List.of(
6             "Andry", "Miora", "Antso", "Fifaliana", "Koloina",
7             "Mathias", "Bako", "Ainatiana"
8         );
9
10        System.out.println("# 1) tous les noms");
11        noms.stream()
12            .forEach(nom -> System.out.println(nom));
13
14        System.out.println("# 2) noms commençant par A");
15        noms.stream()
16            .filter(nom -> nom.startsWith("A"))
17            .forEach(nom -> System.out.println(nom));
18
19        System.out.println("# 3) noms en majuscules");
20        noms.stream()
21            .map(nom -> nom.toUpperCase())
22            .forEach(nom -> System.out.println(nom));
23
24        System.out.println("# 4) noms triés par ordre alphabétique");
25        noms.stream()
26            .sorted()
27            .forEach(nom -> System.out.println(nom));
28
29        System.out.println("# 5) nombre de noms de plus de 5 caractères");
30        long nbLongs = noms.stream()
31            .filter(nom -> nom.length() > 5)
32            .count();
33        System.out.println("Nombre de noms de plus de 5 caractères : " + nbLongs);
34    }
}
```

35 }

Sortie obtenue

```
# 1) tous les noms
Andry
Miora
Antso
Fifaliana
Koloina
Mathias
Bako
Ainatiana
# 2) noms commençant par A
Andry
Antso
Ainatiana
# 3) noms en majuscules
ANDRY
MIORA
ANTSO
FIFALIANA
KOLOINA
MATHIAS
BAKO
AINATIANA
# 4) noms triés par ordre alphabétique
Ainatiana
Andry
Antso
Bako
Fifaliana
Koloina
Mathias
Miora
# 5) nombre de noms de plus de 5 caractères
Nombre de noms de plus de 5 caractères : 4
```

Piège classique

Chaque question relance `noms.stream()` : un `Stream` ne se consomme qu'une seule fois. Réutiliser une variable de type `Stream<String>` d'une question à l'autre lève une `IllegalStateException` (« stream has already been operated upon or closed »).

3. Exercice 3 — Streams sur une liste de notes

Énoncé

Sur une liste de notes : afficher les notes ≥ 10 , compter les admis, calculer la moyenne, afficher la meilleure note, trier les notes par ordre décroissant.

Solution

```
1 import java.util.List;
```

```
2 import java.util.stream.Collectors;
3
4 public class Exercice3 {
5     public static void main(String[] args) {
6         List<Double> notes = List.of(12.5, 8.0, 15.0, 9.5, 17.0, 6.5, 11.0, 19.5);
7
8         System.out.println("# 1) notes >= 10");
9         notes.stream()
10             .filter(note -> note >= 10.0)
11             .forEach(note -> System.out.println(note));
12
13         System.out.println("# 2) nombre d'admis");
14         long nbAdmis = notes.stream()
15             .filter(note -> note >= 10.0)
16             .count();
17         System.out.println("Nombre d'admis : " + nbAdmis);
18
19         System.out.println("# 3) moyenne");
20         double moyenne = notes.stream()
21             .mapToDouble(note -> note)
22             .average()
23             .orElse(0.0);
24         System.out.printf("Moyenne : %.2f%n", moyenne);
25
26         System.out.println("# 4) meilleure note");
27         notes.stream()
28             .mapToDouble(note -> note)
29             .max()
30             .ifPresent(meilleure -> System.out.println("Meilleure note : " + meilleure));
31
32         System.out.println("# 5) notes triées par ordre décroissant");
33         notes.stream()
34             .sorted((n1, n2) -> Double.compare(n2, n1))
35             .forEach(note -> System.out.println(note));
36     }
37 }
```

Sortie obtenue

```
# 1) notes >= 10
12.5
15.0
17.0
11.0
19.5
# 2) nombre d'admis
Nombre d'admis : 5
# 3) moyenne
Moyenne : 12.38
# 4) meilleure note
Meilleure note : 19.5
# 5) notes triées par ordre décroissant
19.5
17.0
15.0
12.5
11.0
9.5
```

8.0
6.5

À retenir

`average()` et `max()` renvoient un `OptionalDouble` (ou `Optional<T>`) : jamais une valeur brute directement. Deux façons propres de le débiller sans laisser de `Optional[...]` à l'écran : `.orElse(0.0)` pour continuer un calcul, ou `.ifPresent(valeur -> ...)` pour ne rien afficher s'il n'y a pas de résultat.

4. Exercice 4 — Classe Produit et Collectors

Énoncé

Sur une collection de `Produit(nom, categorie, prix)` : afficher les produits de `prix > 10 000`, les trier par prix décroissant, calculer le prix moyen, regrouper par catégorie, trouver le produit le plus cher.

Solution

`Produit` est modélisé en record : trois champs, aucun accesseur ni constructeur à écrire à la main.

```

1 import java.util.List;
2 import java.util.Map;
3 import java.util.stream.Collectors;
4
5 public class Exercice4 {
6
7     record Produit(String nom, String categorie, double prix) {}
8
9     public static void main(String[] args) {
10         List<Produit> produits = List.of(
11             new Produit("Vary gasy", "Alimentation", 8000),
12             new Produit("Ravitoto sy henakisoa", "Alimentation", 15000),
13             new Produit("Lamba landy", "Artisanat", 45000),
14             new Produit("Vakoka mihogo", "Artisanat", 12500),
15             new Produit("Kafe gasy", "Alimentation", 9500),
16             new Produit("Sombily", "Alimentation", 22000),
17             new Produit("Sokay", "Artisanat", 6000)
18         );
19
20         System.out.println("# 1) produits de prix > 10000");
21         produits.stream()
22             .filter(produit -> produit.prix() > 10000)
23             .forEach(produit -> System.out.println(produit.nom() + " (" + produit.categorie() + ") - "
24                 + produit.prix() + " Ar"));
25
26         System.out.println("# 2) produits triés par prix décroissant");
27         produits.stream()
28             .sorted((p1, p2) -> Double.compare(p2.prix(), p1.prix()))
29             .forEach(produit -> System.out.println(produit.nom() + " - " + produit.prix() + " Ar"));
30
31         System.out.println("# 3) prix moyen");
32         double prixMoyen = produits.stream()
33             .mapToDouble(Produit::prix)
34             .average()
35             .orElse(0.0);

```

```

35     System.out.printf("Prix moyen : %.2f Ar%n", prixMoyen);
36
37     System.out.println("# 4) produits regroupés par catégorie");
38     Map<String, List<Produit>> parCategorie = produits.stream()
39         .collect(Collectors.groupingBy(Produit::categorie));
40     parCategorie.forEach((categorie, liste) -> {
41         System.out.println(categorie + " :");
42         liste.forEach(produit -> System.out.println("   " + produit.nom()));
43     });
44
45     System.out.println("# 5) produit le plus cher");
46     produits.stream()
47         .max((p1, p2) -> Double.compare(p1.prix(), p2.prix()))
48         .ifPresent(produit -> System.out.println("Produit le plus cher : " + produit.nom() + " - "
49             + produit.prix() + " Ar"));
50 }

```

Sortie obtenue

```

# 1) produits de prix > 10000
Ravitoto sy henakisoa (Alimentation) - 15000.0 Ar
Lamba landy (Artisanat) - 45000.0 Ar
Vakoka mihogo (Artisanat) - 12500.0 Ar
Sombily (Alimentation) - 22000.0 Ar
# 2) produits triés par prix décroissant
Lamba landy - 45000.0 Ar
Sombily - 22000.0 Ar
Ravitoto sy henakisoa - 15000.0 Ar
Vakoka mihogo - 12500.0 Ar
Kafe gasy - 9500.0 Ar
Vary gasy - 8000.0 Ar
Sokay - 6000.0 Ar
# 3) prix moyen
Prix moyen : 16857.14 Ar
# 4) produits regroupés par catégorie
Artisanat :
    Lamba landy
    Vakoka mihogo
    Sokay
Alimentation :
    Vary gasy
    Ravitoto sy henakisoa
    Kafe gasy
    Sombily
# 5) produit le plus cher
Produit le plus cher : Lamba landy - 45000.0 Ar

```

À retenir

Le regroupement par catégorie tient en une seule instruction : `Collectors.groupingBy(Produit::categorie)` renvoie directement une `Map<String, List<Produit>>`, sans boucle ni structure intermédiaire à gérer soi-même.

Ordre des catégories

groupingBy utilise par défaut une HashMap : l'ordre d'affichage des catégories (ici Artisanat avant Alimentation) dépend du hachage des clés, pas de l'ordre d'apparition dans la liste source. Pour un ordre garanti, passer `Collectors.groupingBy(Produit::categorie, TreeMap::new, Collectors.toList())`.

5. Exercice 5 — Synthèse : gestion d'étudiants**Énoncé**

Sur une collection de `Etudiant(matricule, nom, prenom, niveau, moyenne)` : afficher les admis, afficher les non admis, trier par moyenne décroissante, calculer la moyenne générale, regrouper par niveau, afficher le meilleur étudiant. Admis et non admis doivent être obtenus par un seul `partitioningBy`.

Solution

```

1 import java.util.List;
2 import java.util.Map;
3 import java.util.stream.Collectors;
4
5 public class Exercice5 {
6
7     record Etudiant(String matricule, String nom, String prenom, int niveau, double moyenne) {}
8
9     public static void main(String[] args) {
10         List<Etudiant> etudiants = List.of(
11             new Etudiant("2023-001", "Rakoto", "Andry", 2, 15.5),
12             new Etudiant("2023-002", "Rabe", "Mathias", 3, 12.0),
13             new Etudiant("2023-003", "Rajaonarivelo", "Koloina", 1, 18.0),
14             new Etudiant("2023-004", "Rakotomalala", "Fifaliana", 2, 14.0),
15             new Etudiant("2023-005", "Raso", "Antso", 3, 16.5),
16             new Etudiant("2023-006", "Rajaonarivelo", "Miora", 1, 19.0),
17             new Etudiant("2023-007", "Randriamampionona", "Bako", 2, 7.0)
18         );
19
20         System.out.println("# 1) et 2) admis / non admis, en un seul partitioningBy");
21         Map<Boolean, List<Etudiant>> partition = etudiants.stream()
22             .collect(Collectors.partitioningBy(etudiant -> etudiant.moyenne() >= 10.0));
23
24         System.out.println("-- Admis --");
25         partition.get(true)
26             .forEach(etudiant -> System.out.println(etudiant.nom() + " " + etudiant.prenom() + " -
27                 Moyenne : " + etudiant.moyenne()));
28
29         System.out.println("-- Non admis --");
30         partition.get(false)
31             .forEach(etudiant -> System.out.println(etudiant.nom() + " " + etudiant.prenom() + " -
32                 Moyenne : " + etudiant.moyenne()));
33
34         System.out.println("# 3) étudiants triés par moyenne décroissante");
35         etudiants.stream()
36             .sorted((e1, e2) -> Double.compare(e2.moyenne(), e1.moyenne()))
37             .forEach(etudiant -> System.out.println(etudiant.nom() + " " + etudiant.prenom() + " -
38                 Moyenne : " + etudiant.moyenne()));
39     }
40 }

```



```

37     System.out.println("# 4) moyenne générale");
38     double moyenneGenerale = etudiants.stream()
39         .mapToDouble(Etudiant::moyenne)
40         .average()
41         .orElse(0.0);
42     System.out.printf("Moyenne générale : %.2f%n", moyenneGenerale);
43
44     System.out.println("# 5) étudiants regroupés par niveau");
45     Map<Integer, List<Etudiant>> parNiveau = etudiants.stream()
46         .collect(Collectors.groupingBy(Etudiant::niveau));
47     parNiveau.forEach((niveau, liste) -> {
48         System.out.println("Niveau " + niveau + " :");
49         liste.forEach(etudiant -> System.out.println("    " + etudiant.nom() + " " + etudiant.prenom
50             () + " - Moyenne : " + etudiant.moyenne()));
51     });
52
53     System.out.println("# 6) meilleur étudiant");
54     etudiants.stream()
55         .max((e1, e2) -> Double.compare(e1.moyenne(), e2.moyenne()))
56         .ifPresent(etudiant -> System.out.println("Meilleur étudiant : " + etudiant.nom() + " " +
57             etudiant.prenom() + " - Moyenne : " + etudiant.moyenne()));

```

Sortie obtenue

```

# 1) et 2) admis / non admis, en un seul partitioningBy
-- Admis --
Rakoto Andry - Moyenne : 15.5
Rabe Mathias - Moyenne : 12.0
Rajaonarivelo Koloina - Moyenne : 18.0
Rakotomalala Fifaliana - Moyenne : 14.0
Raso Antso - Moyenne : 16.5
Rajaonarivelo Miora - Moyenne : 19.0
-- Non admis --
Randriamampionona Bako - Moyenne : 7.0
# 3) étudiants triés par moyenne décroissante
Rajaonarivelo Miora - Moyenne : 19.0
Rajaonarivelo Koloina - Moyenne : 18.0
Raso Antso - Moyenne : 16.5
Rakoto Andry - Moyenne : 15.5
Rakotomalala Fifaliana - Moyenne : 14.0
Rabe Mathias - Moyenne : 12.0
Randriamampionona Bako - Moyenne : 7.0
# 4) moyenne générale
Moyenne générale : 14.57
# 5) étudiants regroupés par niveau
Niveau 1 :
    Rajaonarivelo Koloina - Moyenne : 18.0
    Rajaonarivelo Miora - Moyenne : 19.0
Niveau 2 :
    Rakoto Andry - Moyenne : 15.5
    Rakotomalala Fifaliana - Moyenne : 14.0
    Randriamampionona Bako - Moyenne : 7.0
Niveau 3 :
    Rabe Mathias - Moyenne : 12.0
    Raso Antso - Moyenne : 16.5
# 6) meilleur étudiant

```

Meilleur étudiant : Rajaonarivelo Miora - Moyenne : 19.0

À retenir

`Collectors.partitioningBy(predicat)` renvoie toujours une `Map<Boolean, List<Etudiant>>` à exactement deux clés, `true` et `false` — contrairement à `groupingBy`, ce n'est jamais vide même si un des deux groupes est vide. C'est le bon outil dès qu'un critère est binaire (admis/non admis, réussite/échec), là où `groupingBy` convient à un critère à plusieurs valeurs (le niveau, ici en question 5).

Piège de comparateur

Pour trier par moyenne *décroissante* (question 3), inverser l'ordre des opérandes dans `Double.compare(e2.moyenne(), e1.moyenne())` plutôt que de négater le résultat d'un `Double.compare(e1, e2)` : les deux fonctionnent, mais l'inversion des opérandes reste la version la plus lisible et évite tout risque de débordement lié à une négation d'entier.

Synthèse

Le tableau ci-dessous récapitule l'opération Stream centrale de chaque question, tous exercices confondus.

Exercice	Besoin	Opération clé
Exercice 1	Comportement paramétrable	interface fonctionnelle + lambda
Exercice 2	Filtrer, transformer, trier, compter	<code>filter</code> , <code>map</code> , <code>sorted</code> , <code>count</code>
Exercice 3	Agréger des nombres	<code>mapToDouble</code> , <code>average</code> , <code>max</code>
Exercice 4	Regrouper des objets métier	<code>Collectors.groupingBy</code>
Exercice 5	Partager en deux groupes	<code>Collectors.partitioningBy</code>